Shell Lab 实验报告

一、实验简介

1.1 实验目的

本实验旨在通过编写一个支持作业控制（Job Control）的简易 Unix Shell 程序，深入理解现代操作系统中进程控制和信号处理的核心概念。具体目标包括：

1. 掌握 Unix/Linux 进程管理机制，包括 `fork`、`execve` 等系统调用的使用
2. 理解信号（Signal）的发送、接收和处理机制，特别是 SIGCHLD、SIGINT、SIGTSTP
3. 掌握作业控制（Job Control）的实现原理，包括前台/后台进程组的隔离
4. 理解并发编程中的竞争条件（Race Condition）及其通过信号阻塞（Signal Masking）来规避的方法
5. 熟悉 Shell 程序的基本工作原理，为主线课程 CS:APP 第八章"异常控制流"提供实践支撑

1.2 实验环境

使用 Docker 拉取旧版 Ubuntu 镜像进行实验，降低出现环境问题的风险。

项目	配置信息
宿主机操作系统	Ubuntu 24.04
容器操作系统	Ubuntu 14.04
GCC 版本	4.8.4
Perl 版本	5.18.2
编译器标志	-Wall -O2

```
long@ubuntu2404:~$ docker ps
CONTAINER ID   IMAGE          COMMAND       CREATED       STATUS             PORTS     NAMES
8eaaaae95811   ns2-image:v1   "/bin/bash"   12 days ago   Up About a minute            shell-lab
long@ubuntu2404:~$ docker exec -it shell-lab  /bin/bash
root@8eaaaae95811:/# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 14.04.6 LTS
Release:        14.04
Codename:       trusty
root@8eaaaae95811:/#
```

1.3 实验资源

文件	说明
`tsh.c`	Shell 主程序源文件，需要补全 7 个函数
`Makefile`	编译和测试规则
`sdriver.pl`	Shell 测试驱动程序（Perl 脚本）
`tshref.out`	参考实现对所有 16 个 trace 的完整输出
`trace01.txt ~ trace16.txt`	16 个测试用例，难度递增
`myspin.c`	测试辅助程序：自旋等待 N 秒
`mysplit.c`	测试辅助程序：fork 子进程后自旋（用于验证进程组信号）
`mystop.c`	测试辅助程序：自旋后向自身发送 SIGTSTP
`myint.c`	测试辅助程序：自旋后向自身发送 SIGINT

二、实验过程

本实验按照 trace01 到 trace16 的顺序逐步推进。16 个 trace 本身就是按实现难度递增设计的，每个 trace 驱动一个或一组函数的实现。以下按实际开发顺序记录每一步的思考、实现和验证过程。

2.1 trace01-02：让 Shell 能启动和退出

目标：Shell 进程能正常启动、收到 `quit` 命令时正常退出。
涉及函数：`builtin_cmd`
思考与分析：
trace01 只发送 `CLOSE` + `WAIT`，测试 EOF 退出。`main` 函数中 `feof(stdin)` 的处理已被框架提供，空壳也能通过。
trace02 发送 `quit` + `WAIT`，要求 Shell 认识 `quit` 这个内置命令并调用 `exit(0)`。当前 `builtin_cmd` 是一个空壳（`return 0`），表示不认识任何命令。需要在此函数中添加对 `quit` 的识别。
`builtin_cmd` 的执行逻辑非常简单：接收一个字符串数组 `argv`（`argv[0]` 是命令名），如果是内置命令就执行并返回 1，否则返回 0 让 `eval` 按外部命令处理。
实现：

```
int builtin_cmd(char **argv)
{
    if (!strcmp(argv[0], "quit"))
        exit(0);
    return 0;
}
```

此时 Shell 只认识 `quit`，其他所有输入都被视为"不是内置命令"，但由于 `eval` 未实现，还无法执行外部程序。
测试验证：

```
root@8eaaaae95811:~/work# make test01
./sdriver.pl -t trace01.txt -s ./tsh -a "-p"
#
# trace01.txt - Properly terminate on EOF.
#
root@8eaaaae95811:~/work# make test02
./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
#
# trace02.txt - Process builtin quit command.
#
root@8eaaaae95811:~/work#
```

2.2 trace03：实现前台命令执行

目标：Shell 能执行 `/bin/echo` 等简单的外部命令（前台运行）。
涉及函数：`eval`、`waitfg`、`sigchld_handler`
思考与分析：
trace03 的内容是 `/bin/echo tsh> quit` 之后 `quit`。要在 tsh 里执行 `/bin/echo`，需要一个完整的"fork → execve → wait"流程：

1. **父进程 fork 子进程**：Shell 自己不能变成 echo——那样 Shell 就没了。必须 fork 一个新进程。
2. **子进程通过 execve 加载目标程序**：用 echo 程序替换子进程的内存映像。
3. **父进程等待子进程结束**：前台命令必须阻塞 Shell，不让用户输入新命令。
4. **回收子进程防止僵尸**：子进程终止后必须由父进程调用 `waitpid` 回收。

关键点：**waitfg 不调用 waitpid，只等待** `fgpid(jobs)` 变为零。子进程的实际回收由 `sigchld_handler` **统一负责**。

实现：

```
void eval(char *cmdline)
{
    char *argv[MAXARGS];
    int bg = parseline(cmdline, argv);
    pid_t pid;

    if (argv[0] == NULL)
        return;

    if (!builtin_cmd(argv)) {
```

```
        if ((pid = fork()) == 0) {
            // 子进程
            setpgid(0, 0); // 独立出来进程组
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found\n", argv[0]);
            }
            exit(0);
        }
        // 父进程
        addjob(jobs, pid, FG, cmdline);
        waitfg(pid);
    }
}

void waitfg(pid_t pid)
{
    while (fgpid(jobs) == pid)
        sleep(1);
}

void sigchld_handler(int sig)
{
    int olderrno = errno;
    int status;
    pid_t pid;

    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        if (WIFEXITED(status))
            deletejob(jobs, pid);
    }
    errno = olderrno;
}
```

`eval` 中 fork 子进程 → `setpgid(0,0)` 创建独立进程组 → `execve` 加载程序。父进程 `addjob` 后调用 `waitfg` 阻塞等待。`waitfg` 不调 `waitpid`，而是轮询 `fgpid(jobs)`：当 `sigchld_handler` 回收子进程并 `deletejob` 后，`fgpid` 不再返回该 pid，循环退出。
`sigchld_handler` 当前只有基础版：仅处理 `WIFEXITED`（正常退出）后静默删除。

测试验证：

```
root@8eaaaae95811:~/work# make test03
./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
root@8eaaaae95811:~/work#
```

2.3 trace04-05：后台作业与 jobs 命令

目标：支持 `&` 后台运行、`jobs` 命令列出作业。
涉及函数：`eval`、`builtin_cmd`
思考与分析：
trace04 发 `./myspin 1 &`，Shell 应打印 `[1] (PID) ./myspin 1 &` 后立即回到提示符——不等待子进程。trace05 发多个后台命令后发 `jobs`，要求列出所有活跃作业及其状态。
需要在 `eval` 中增加前后台分支：`parseline` 已经解析了 `&` 并通过返回值告知是后台作业，只需要在父进程中根据该标志决定"waitfg 等待"还是"打印信息后直接返回"。
`jobs` 命令调用框架提供的 `listjobs(jobs)` 即可。
实现：
`eval` 中增加后台分支：

```
void eval(char *cmdline)
{
    char *argv[MAXARGS];
    int bg = parseline(cmdline, argv);
    pid_t pid;

    if (argv[0] == NULL)
        return;

    if (!builtin_cmd(argv)) {
        if ((pid = fork()) == 0) {
            setpgid(0, 0);
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found\n", argv[0]);
            }
            exit(0);
        }

        addjob(jobs, pid, (bg ? BG : FG), cmdline);

        if (!bg) {
            waitfg(pid);       // 前台：等待
        } else {
            // 后台：打印作业信息后直接返回
            printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline);
        }
    }
}
```

`builtin_cmd` 中增加 `jobs` 命令：

```
int builtin_cmd(char **argv)
{
    if (!strcmp(argv[0], "quit"))
        exit(0);
```

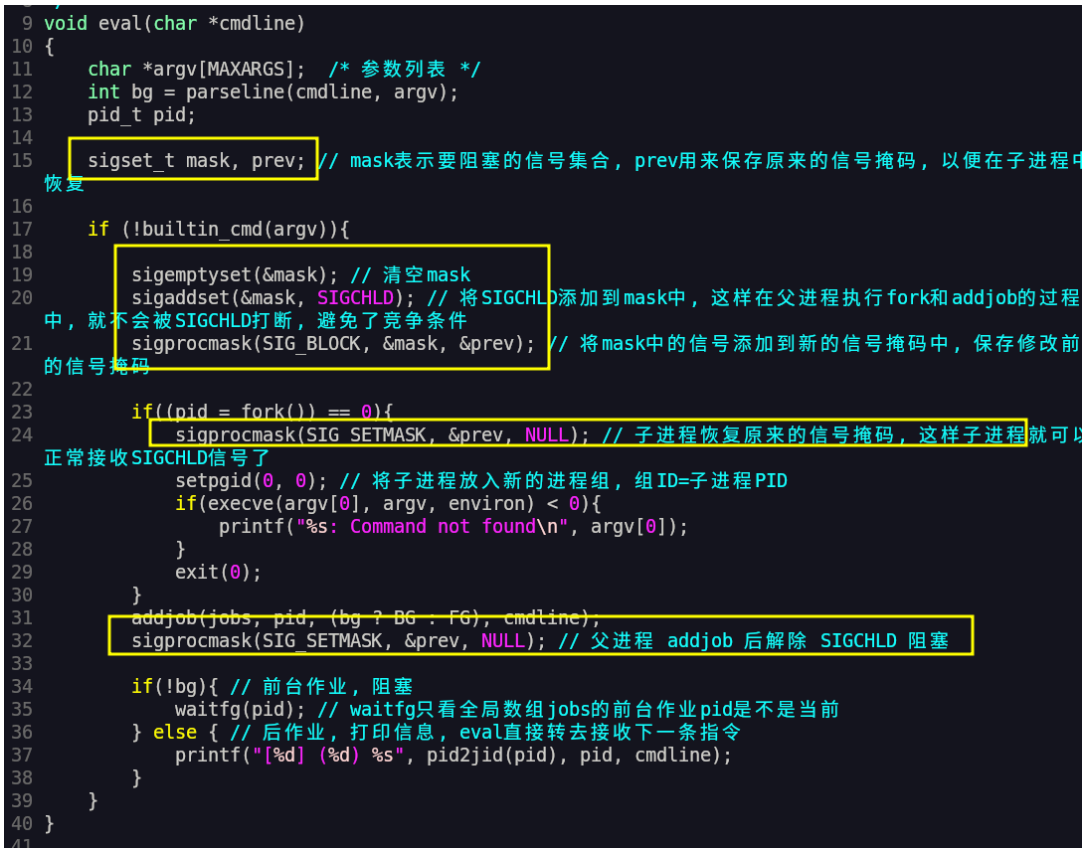
```
    if (!strcmp(argv[0], "jobs")) {
        listjobs(jobs);
        return 1;
    }
    if (!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg")) {
        do_bgfg(argv);          // 先注册，具体实现后面
        return 1;
    }
    return 0;
}
```

测试验证：

```
root@8eaaaae95811:~/work# make test04
./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (65) ./myspin 1 &
root@8eaaaae95811:~/work# make test05
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (71) ./myspin 2 &
tsh> ./myspin 3 &
[2] (73) ./myspin 3 &
tsh> jobs
[1] (71) Running ./myspin 2 &
[2] (73) Running ./myspin 3 &
root@8eaaaae95811:~/work#
```

2.4 补上信号阻塞 —— 修复竞争条件

目标：消除 fork 和 addjob 之间的竞争条件。
涉及函数：`eval`（增加 `sigprocmask` 阻塞）
思考与分析：
在 2.2 节的实现中存在一个隐藏 bug：父进程 fork 后，子进程可能瞬间执行完毕并终止，内核在父进程还没来得及调用 `addjob` 时就发送 SIGCHLD。`sigchld_handler` 被调用，尝试 `deletejob(pid)`——但此时这个 pid 还没被 `addjob` 加入 `jobs[]` 数组，handler 找不到目标。然后 `addjob` 又把这个 pid 加进去了，但 handler 不会再收到 SIGCHLD——这个作业永远留在列表中变僵尸。
解决：在 fork 前用 `sigprocmask(SIG_BLOCK, &mask, &prev)` 阻塞 SIGCHLD，创造一个"安全窗口"。addjob 完成后再 `sigprocmask(SIG_SETMASK, &prev, NULL)` 解除阻塞。期间即使子进程已经终止，SIGCHLD 也被内核标记为挂起（pending），等到解除阻塞后才递送——此时 addjob 已完成，handler 能安全地找到该作业。
子进程也必须处理：子进程通过 fork 继承了父进程被阻塞的信号掩码，必须在 `execve` 前用 `sigprocmask(SIG_SETMASK, &prev, NULL)` 恢复为 fork 前的干净状态。



实现:

```
void eval(char *cmdline)
{
    char *argv[MAXARGS];
    int bg = parseline(cmdline, argv);
    pid_t pid;

    if (argv[0] == NULL)
        return;

    if (!builtin_cmd(argv)) {
        sigset_t mask, prev;

        sigemptyset(&mask);
        sigaddset(&mask, SIGCHLD);
        sigprocmask(SIG_BLOCK, &mask, &prev);  /* ① fork 前阻塞 SIGCHLD */

        if ((pid = fork()) == 0) {
            sigprocmask(SIG_SETMASK, &prev, NULL);  /* ② 子进程恢复掩码 */
            setpgid(0, 0);
            if (execve(argv[0], argv, environ) < 0) {
                printf("%s: Command not found\n", argv[0]);
            }
            exit(0);
        }

        addjob(jobs, pid, (bg ? BG : FG), cmdline);
        sigprocmask(SIG_SETMASK, &prev, NULL);  /* ③ 父进程 addjob 后解除阻塞 */
```

```
        if (!bg) {
            waitfg(pid);
        } else {
            printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline);
        }
    }
}
```

三处 `sigprocmask` 调用：①fork 前阻塞 SIGCHLD，确保 addjob 先于 handler 完成；②子进程恢复 fork 前的干净掩码；③父进程 addjob 后解除阻塞，此时挂起的 SIGCHLD 才被递送。

2.5 trace06-07：Ctrl+C 信号处理

目标：用户按 Ctrl+C 时，终止前台作业但不影响后台作业。
涉及函数：`sigint_handler`、`sigchld_handler`（完整补全）
思考与分析：
trace06 在 `./myspin 4` 前台运行 2 秒后发送 `INT`。期望输出 `Job [1] (PID) terminated by signal 2`。trace07 验证只有前台作业被杀，后台作业依然 Running。

- **`sigint_handler`**（转发者）：获取前台作业 PID → `kill(-pid, SIGINT)` 向整个进程组发信号。注意必须用 `-pid`（负号 = 进程组）而非 `pid`（单进程），因为 `mysplit` 等程序会创建子进程。只打印消息，不回收。
- **`sigchld_handler`**（善后者）：检测到 `WIFSIGNALED` 后打印终止消息 + `deletejob`；检测到 `WIFSTOPPED` 后打印停止消息 + 改状态为 ST；检测到 `WIFEXITED` 后静默删除。必须同时使用 `WNOHANG`（非阻塞）和 `WUNTRACED`（检测停止），用 `while` 循环一次性处理所有状态变更的子进程。

为什么 handler 不直接 `deletejob`？因为 `kill` 只是**发送**信号，子进程什么时候真正终止由内核决定。如果 handler 发了 kill 后立即 deletejob，子进程可能还没死，导致 `jobs[]` 状态不一致。正确流程是：转发者发信号 → 内核杀进程 → 内核发 SIGCHLD → 善后者 `waitpid` 确认已死 → 安全清理。

实现：

```
void sigint_handler(int sig)
{
    pid_t pid = fgpid(jobs);
    if (pid != 0)
        kill(-pid, SIGINT);  // 负号 = 向整个进程组发信号
}

void sigchld_handler(int sig)
{
    int olderrno = errno;
    int status;
    pid_t pid;
    struct job_t *job;
```

```
    while ((pid = waitpid(-1, &status, WNOHANG | WUNTRACED)) > 0) {
        if (WIFEXITED(status)) {
            deletejob(jobs, pid);               // 正常终止
        }
        else if (WIFSIGNALED(status)) {
            printf("Job [%d] (%d) terminated by signal %d\n",
                   pid2jid(pid), pid, WTERMSIG(status));
            deletejob(jobs, pid);               // 被信号杀死，打印后删除
        }
        else if (WIFSTOPPED(status)) {
            printf("Job [%d] (%d) stopped by signal %d\n",
                   pid2jid(pid), pid, WSTOPSIG(status));
            job = getjobpid(jobs, pid);
            if (job != NULL) job->state = ST;   // 被信号停止，保留在列表
        }
    }
    errno = olderrno;
}
```

`sigint_handler` **只负责转发**——获取前台作业 PID，`kill(-pid, SIGINT)` 向整个进程组发信号。消息打印和作业回收由 `sigchld_handler` 负责。

`sigchld_handler` 三个分支：`WIFEXITED` 静默删除；`WIFSIGNALED` 打印终止消息后删除；`WIFSTOPPED` 打印停止消息后改为 ST 状态保留。使用 `WNOHANG`（非阻塞）+ `WUNTRACED`（检测停止），`while` 循环一次性处理所有状态变更的子进程。

测试验证：

```
root@8eaaaae95811:~/work# make test06
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (80) terminated by signal 2
root@8eaaaae95811:~/work# make test07
./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (86) ./myspin 4 &
tsh> ./myspin 5
Job [2] (88) terminated by signal 2
tsh> jobs
[1] (86) Running ./myspin 4 &
root@8eaaaae95811:~/work#
```

2.6 trace08：Ctrl+Z 信号处理

目标：用户按 Ctrl+Z 时，挂起前台作业，后台作业不受影响。
涉及函数：`sigtstp_handler`
思考与分析：

trace08 与 trace07 结构完全对称——用 `TSTP` 替代 `INT`。`sigtstp_handler` 的实现与 `sigint_handler` 完全对称：获取前台作业 PID → `kill(-pid, SIGTSTP)` 向进程组发信号。
子进程收到 SIGTSTP 后停止，内核发 SIGCHLD，`sigchld_handler` 的 `WIFSTOPPED` 分支处理：打印 `stopped by signal 20`，将状态改为 ST 保留在列表中（不删除）。

实现：

```
void sigtstp_handler(int sig)
{
    pid_t pid = fgpid(jobs);
    if (pid != 0)
        kill(-pid, SIGTSTP);
}
```

与 `sigint_handler` 完全对称：获取前台作业 PID，`kill(-pid, SIGTSTP)` 向进程组发信号。子进程停止后，内核发 SIGCHLD，`sigchld_handler` 的 `WIFSTOPPED` 分支处理后续的打印和状态变更。

测试验证：

```
root@8eaaaae95811:~/work# make test08
./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (95) ./myspin 4 &
tsh> ./myspin 5
Job [2] (97) stopped by signal 20
tsh> jobs
[1] (95) Running ./myspin 4 &
[2] (97) Stopped ./myspin 5
root@8eaaaae95811:~/work#
```

2.7 trace09-10：bg 和 fg 作业控制命令

目标：实现 `bg %JID` 将停止的作业恢复到后台运行，`fg %JID` 将作业切换到前台。
涉及函数：`do_bgfg`
思考与分析：
`do_bgfg` 是最后实现的函数，因为它依赖前面所有的基础设施（`waitfg`、`sigchld_handler`、进程组机制）。
核心流程：

1. **参数解析**：支持 `%JID`（以 `%` 开头）和 PID（纯数字）两种形式，三种错误（无参数、格式错、作业不存在）
2. **发送 SIGCONT**：`kill(-(job->pid), SIGCONT)` 唤醒**整个进程组**（负号确保 `mysplit` 等多进程作业全部唤醒）

3. **改状态并处理**：bg → state=BG + 打印作业信息后立即返回；fg → state=FG + `waitfg` 阻塞等待

实现：

```
void do_bgfg(char **argv)
{
    struct job_t *job;
    char *cmd = argv[0];
    char *arg = argv[1];
    int jid;
    pid_t pid;

    if (arg == NULL) {
        printf("%s command requires PID or %%jobid argument\n", cmd);
        return;
    }

    if (arg[0] == '%') {      // JID 形式
        jid = atoi(&arg[1]);
        job = getjobjid(jobs, jid);
        if (job == NULL) {
            printf("%%%d: No such job\n", jid);
            return;
        }
    } else if (isdigit(arg[0])) {    // PID 形式
        pid = atoi(arg);
        job = getjobpid(jobs, pid);
        if (job == NULL) {
            printf("(%d): No such process\n", pid);
            return;
        }
    } else {
        printf("%s: argument must be a PID or %%jobid\n", cmd);
        return;
    }

    kill(-(job->pid), SIGCONT);  // 向进程组发送 SIGCONT 唤醒所有进程

    if (!strcmp(cmd, "bg")) {
        job->state = BG;
        printf("[%d] (%d) %s", job->jid, job->pid, job->cmdline);
    } else {                     // "fg"
        job->state = FG;
        waitfg(job->pid);
    }
}
```

核心流程：参数解析（`%JID` / `PID` /非法参数三条分支）→ `kill(-pid, SIGCONT)` 唤醒进程组 → bg 改状态为 BG 并打印后返回，fg 改状态为 FG 并 `waitfg` 阻塞等待。

测试验证：

```
root@8eaaaae95811:~/work# make test09
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (104) ./myspin 4 &
tsh> ./myspin 5
Job [2] (106) stopped by signal 20
tsh> jobs
[1] (104) Running ./myspin 4 &
[2] (106) Stopped ./myspin 5
tsh> bg %2
[2] (106) ./myspin 5
tsh> jobs
[1] (104) Running ./myspin 4 &
[2] (106) Running ./myspin 5
root@8eaaaae95811:~/work# make test10
./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (115) ./myspin 4 &
tsh> fg %1
Job [1] (115) stopped by signal 20
tsh> jobs
[1] (115) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
root@8eaaaae95811:~/work#
```

2.8 trace11-13：进程组信号验证

目标：验证 `kill(-pid, sig)` 对整个进程组生效（而非单个进程）。
涉及函数：无需新代码，验证现有实现的正确性
思考与分析：
这三个 trace 使用 `./mysplit 4`（会 fork 子进程，父子同进程组）来验证进程组信号机制：

- **trace11**：SIGINT 向前台进程组，`/bin/ps a` 确认父子进程都被终止，无残留
- **trace12**：SIGTSTP 向前台进程组，ps 显示父子进程都处于 `T`（停止）状态
- **trace13**：`fg %1` 恢复整个进程组（通过 `kill(-pid, SIGCONT)`+进程组），ps 确认进程恢复正常运行

如果代码中误用 `kill(pid, SIGINT)`（没写负号），则只有父进程被杀，子进程变孤儿继续跑，`/bin/ps a` 会看到残留进程——这三个 trace 会暴露该 bug。

测试验证：

```
root@8eaaaae95811:~/work# make test11
./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (125) terminated by signal 2
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 /bin/bash
     17 pts/1    Ss     0:00 /bin/bash
    120 pts/1    S+     0:00 make test11
    121 pts/1    S+     0:00 /bin/sh -c ./sdriver.pl -t trace11.txt -s ./tsh -
a "-p"
    122 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./ts
h -a -p
    123 pts/1    S+     0:00 ./tsh -p
    128 pts/1    R      0:00 /bin/ps a
```

```
root@8eaaaae95811:~/work# make test12
./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (144) stopped by signal 20
tsh> jobs
[1] (144) Stopped ./mysplit 4
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 /bin/bash
     17 pts/1    Ss     0:00 /bin/bash
    139 pts/1    S+     0:00 make test12
    140 pts/1    S+     0:00 /bin/sh -c ./sdriver.pl -t trace12.txt -s ./tsh -
a "-p"
    141 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./ts
h -a -p
    142 pts/1    S+     0:00 ./tsh -p
    144 pts/1    T      0:00 ./mysplit 4
    145 pts/1    T      0:00 ./mysplit 4
    148 pts/1    R      0:00 /bin/ps a
```

```
root@8eaaaae95811:~/work# make test13
./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (154) stopped by signal 20
tsh> jobs
[1] (154) Stopped ./mysplit 4
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 /bin/bash
     17 pts/1    Ss     0:00 /bin/bash
    149 pts/1    S+     0:00 make test13
    150 pts/1    S+     0:00 /bin/sh -c ./sdriver.pl -t trace13.txt -s ./tsh -
a "-p"
    151 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./ts
h -a -p
    152 pts/1    S+     0:00 ./tsh -p
    154 pts/1    T      0:00 ./mysplit 4
    155 pts/1    T      0:00 ./mysplit 4
    158 pts/1    R      0:00 /bin/ps a
tsh> fg %1
tsh> /bin/ps a
    PID TTY      STAT   TIME COMMAND
      1 pts/0    Ss+    0:00 /bin/bash
     17 pts/1    Ss     0:00 /bin/bash
    149 pts/1    S+     0:00 make test13
    150 pts/1    S+     0:00 /bin/sh -c ./sdriver.pl -t trace13.txt -s ./tsh -
a "-p"
    151 pts/1    S+     0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./ts
h -a -p
    152 pts/1    S+     0:00 ./tsh -p
    161 pts/1    R      0:00 /bin/ps a
```

2.9 trace14：错误处理

目标：各种异常输入都能正确处理，不崩溃。
涉及函数：`eval`（Command not found）、`do_bgfg`（参数校验）
思考与分析：
trace14 覆盖了以下错误场景：

输入	错误类型	期望输出
`./bogus`	不存在的命令	`./bogus: Command not found`
`fg` / `bg`	缺少参数	`fg/bg command requires PID or %jobid argument`
`fg a` / `bg a`	非法参数格式	`fg/bg: argument must be a PID or %jobid`
`fg 9999999` / `bg 9999999`	不存在的 PID	`(9999999): No such process`
`fg %2` / `bg %2`	不存在的 JID	`%2: No such job`

这些错误处理不需要额外代码——`Command not found` 在 `eval` 的 `execve` 失败分支中已实现，参数校验在 `do_bgfg` 的参数解析逻辑中已覆盖。

测试验证：

```
root@8eaaaae95811:~/work# make test14
./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (169) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 9999999
(9999999): No such process
tsh> bg 9999999
(9999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (169) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (169) ./myspin 4 &
tsh> jobs
[1] (169) Running ./myspin 4 &
root@8eaaaae95811:~/work#
```

2.10 trace15-16：综合回归测试

目标：所有功能协同工作，复杂场景下状态转换正确。
涉及函数：全部 7 个函数
思考与分析：
trace15 串联了所有功能：错误命令 → SIGINT 杀前台 → 后台多作业 → fg 切换 → TSTP 停止 → bg 恢复 → `%3` 不存在的错误处理 → bg 成功 → fg 等待完成 → quit 退出。
trace16 验证子进程**自身**发出的信号（`mystop` 自发送 SIGTSTP、`myint` 自发送 SIGINT），而非终端按键产生的信号。Shell 对此应无差别处理——只要子进程状态变更，`sigchld_handler` 一律回收。

测试验证：

```
root@8eaaaae95811:~/work# make test15
./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (197) terminated by signal 2
tsh> ./myspin 3 &
[1] (199) ./myspin 3 &
tsh> ./myspin 4 &
[2] (201) ./myspin 4 &
tsh> jobs
[1] (199) Running ./myspin 3 &
[2] (201) Running ./myspin 4 &
tsh> fg %1
Job [1] (199) stopped by signal 20
tsh> jobs
[1] (199) Stopped ./myspin 3 &
[2] (201) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (199) ./myspin 3 &
tsh> jobs
[1] (199) Running ./myspin 3 &
[2] (201) Running ./myspin 4 &
tsh> fg %1
tsh> quit
```

```
root@8eaaaae95811:~/work# make test16
./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#     signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (186) stopped by signal 20
tsh> jobs
[1] (186) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (189) terminated by signal 2
root@8eaaaae95811:~/work#
```

三、实验总结

3.1 核心收获

1. **异常控制流（CS:APP Chapter 8）**：Shell 是异常控制流的集中体现——`fork` 创建并发流，`execve` 替换控制流，信号在更高层次改变执行路径。
2. **信号处理与并发**：信号 handler 与主程序共享 `jobs[]` 全局数组，形成并发访问。通过 `sigprocmask` 在 fork 前后阻塞/解除 SIGCHLD，消除了经典的 addjob/deletejob 竞争条件（CS:APP 8.5.6 节）。
3. **进程组与作业控制**：`setpgid(0,0)` 实现进程组隔离，`kill(-pid, sig)` 向整个进程组发送信号，保证多进程作业的原子操作（CS:APP 8.5.3 节）。
4. **关注点分离设计**：`sigint_handler`/`sigtstp_handler` 只转发信号，`sigchld_handler` 只回收善后，`waitfg` 只轮询等待——三者通过 `jobs[]` 数组通信而不互相调用。

3.2 遇到的困难与解决方案

1. **信号竞争条件**：fork 和 addjob 之间的时间窗口可能导致 handler 在 addjob 之前尝试删除作业。通过在 fork 前阻塞 SIGCHLD、addjob 后解除来消除窗口。
2. **sigprocmask 放置位置**：阻塞代码放在 `if(!builtin_cmd)` 外部会导致内置命令永久阻塞 SIGCHLD。应放在外部命令分支内部。
3. **waitfg 与 sigchld_handler 的职责划分**：两者都调 `waitpid` 会互相竞争。采用 handler 负责回收、waitfg 负责轮询的分离方案。
4. **进程组信号的正确使用**：`kill(pid, sig)` 只发单进程，`kill(-pid, sig)` 发全组——`mysplit` 测试暴露了用错前者时子进程变孤儿的 bug。